23AID204 ADVANCED DATA STRUCTURES AND ALGORITHMS

Title: Building a prefix trie based IP routing table for network packet forwarding

Allam Keerthana Chowdary – [CB.AI.U4AID23001]
Kota Kedharnadh – [CB.AI.U4AID23015]
M Sai Krishna – [CB.AI.U4AID23020]
M Rishitha – [CB.AI.U4AID23023]

INTRODUCTION

- Routing is the process of determining the best path for data to travel across a network from the source to the destination.
- Routers make decisions based on the destination IP address to forward packets to the next router or final destination.
- As networks grow in size, efficient routing becomes critical. Slow routing can lead to delays and congestion.
- Finding the best route (shortest path) for each packet is a challenge, especially as the number of routes and IP addresses increases.
- A Trie (Prefix Tree) is a data structure that efficiently stores and searches prefixes (like IP address prefixes).
- It helps in finding the longest matching prefix (LPM), which is crucial for deciding the correct path for a packet.

PROBLEM STATEMENT

- To use the shortest path finding algorithms to fill the routing table then using trie to store the routing table and reduce the lookup time

OBJECTIVES

- To design a network topology with interconnected routers and assign unique IP addresses and subnets to the connections.
- Use Dijkstra's Algorithm to compute the shortest paths between all routers based on the network topology. This will determine the optimal routes for forwarding packets.
- To build a trie for each router's routing table, enabling efficient Longest Prefix Matching (LPM) when a packet arrives with a specific destination IP.

METHODOLOGY

Framework:

- We will design a network with multiple routers connected in topology
- we will create a routing table by using Dijkstra's algorithm to find the shortest path between all routers.
- For each router we build a prefix tree(Trie) that stores the IP prefixes from the routing table. This allows for LPM ,w here we find the most specific route that matches the dest IP.

METHODOLOGY

Algorithm:

- Dijkstra's Algorithm: used to find the shortest path between nodes.
- Trie-based LPM: Trie is tree-like data structures that stores IP prefixes

Paradigm:

Data structures:

trie is used for storing the IP prefixes and to perform LPM.

TRIE:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False
        self.next_hop = None
        self.subnet_mask = None
```

```python
class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, ip_binary, next_hop, subnet_mask):
        node = self.root
        for bit in ip_binary:
            if bit not in node.children:
                node.children[bit] = TrieNode()
            node = node.children[bit]
        node.is_end = True
        node.next_hop = next_hop
        node.subnet_mask = subnet_mask

    def search(self, ip_binary):
        node = self.root
        longest_prefix_match = None
        longest_prefix_hop = None

        for bit in ip_binary:
            if bit in node.children:
                node = node.children[bit]
                if node.is_end:
                    longest_prefix_match = node.subnet_mask
                    longest_prefix_hop = node.next_hop
            else:
                break

        return longest_prefix_hop, longest_prefix_match
```

Trie Operations

Insertion: O(L), where L is the prefix length (≤ 32 for IPv4)

Search: O(L)

Dijkstra's Algorithm
Time Complexity: $O((V + E) \log V)$

V: Number of vertices (routers)
E: Number of edges (connections)

METHODOLOGY

- **Programming Language and Technologies:**

Python is used for Programming.

- **Key Libraries**: NetworkX

A python library for creating and analyzing complex networks and used for running Dijkstra's algorithm.

COMPLETED TASKS

- we were able to store routing tables in trie and were able to find the shortest path using LCP match.

PLAN FOR COMPLETION

Time line for completion : Oct 30th 2024

CONCLUSION

- The use of Trie-based LPM significantly reduces the time required to perform routing lookups.
- The Dijkstra's algorithm ensures optimal routing paths are calculated by minimizing network latency.

REFERENCES

1]J. SMITH, "A FAST TRIE-BASED IP LOOKUP ALGORITHM," M.S. THESIS, DEPT. OF ELECTRONICS ENGINEERING, NEW MEXICO STATE UNIVERSITY-MAIN CAMPUS, LAS CRUCES, NM, USA.

2] T. KIJKANJANARAT AND H. J. CHAO, "FAST IP ROUTING LOOKUPS FOR HIGH PERFORMANCE ROUTERS".